

★ WEEK 16 · PROBLEM SET

Finance Applications: Three Hands-On Problems

Price the Trap · Subtract the Costs · Grade Like a Grader

Each problem: Statement → Principles & Lab Guide → Hand Calculation → Claude Code Prompt

The final set: three pencil-sized audits that catch the three ways projects fail.
Pass your own project through them before anyone else does.

Every Problem Follows the Same Four Steps

Work top to bottom — do not skip the hand calculation, it is where the understanding lives

1 · Statement

the question

A concrete finance scenario with a tiny dataset you can see at a glance.

one slide

2 · Principles

the tools

The formulas involved and a short lab guide: what to compute, in what order.

one slide

3 · Hand Calc

pencil first

Work every number by hand in a table.
Small n , clean arithmetic, no calculator fear.

1–2 slides

4 · Claude Code

scale it up

A metaprompt: paste it into Claude Code and it builds the full lab on real data.

one slide

Hand first, machine second — you check the machine, never the other way around.

Where the Data Comes From

Public / in-script sources first — the final set is nearly all pencil work

Problem	Primary source (public · no signup)	Provided fallback file
P1 · Metrics & luck	confusion counts on the slides	fml_w16_p1_confusion.csv (hand-calc)
P2 · Cost honesty	four numbers on the slides — pure pencil	— (no file needed)
P3 · Rubric	three score rows on the slides	fml_w16_p3_rubric.csv (hand-calc)
Scaled labs	§ 5 in-script set (make_classification)	—

Course rule: if data is free online (yfinance, FRED, OpenML) or generated in-script, the prompt uses it.

P1

PROBLEM 1 · HONEST TEST

Price the Trap, Raise the Bar

One confusion matrix, one search history — the two audits of stage 3

Audit a Fraud Model's Headline

1,000 transactions, 20 frauds. Model: TP 8, FP 2, FN 12, TN 978. It was the best of 20 variants tried; junk scores have sd 0.5

	Predicted fraud	Predicted clean
Actual fraud (20)	TP = 8	FN = 12
Actual clean (980)	FP = 2	TN = 978

Helper

- Dumb baseline: predict 'clean' for everyone.
- Luck bar: $E[\max \text{ of } N \text{ junk}] \approx \text{sd} \times \sqrt{2 \ln N}$; $\sqrt{2 \ln 20} = 2.448$.

Your deliverables

- Model accuracy vs the dumb baseline; precision, recall, F1.
- The luck bar for 20 trials, and a verdict on a best score of 1.2.

The Tools: the Right Metric, Then the Luck Bar

Stage 3 has two questions: is the number honest, and is it luck?

$$F_1 = \frac{2PR}{P+R} \quad \mathbb{E}[\max \widehat{S} \widehat{R}_{\text{junk}}] \approx \sigma_{\widehat{S} \widehat{R}} \sqrt{2 \ln N}$$

Why accuracy lies here

- With 2% positives, 'always clean' scores 98% — for free.
- Accuracy grades the imbalance, not the model (Week 5).
- F1 balances precision and recall — the base rate can't game it.

Lab guide

- Step 1: baseline accuracy, then the model's — compare.
- Step 2: precision 8/10, recall 8/20, then F1.
- Step 3: luck bar 0.5×2.448 ; compare to the reported 1.2.

Two Audits, Two Deflations

The 98.6% deflates to 0.533; the 1.2 deflates to 'luck-level'

Audit	Arithmetic	Result
Dumb baseline accuracy	980 / 1000	98.0%
Model accuracy	(8 + 978) / 1000	98.6% — barely above doing nothing
Precision · Recall · F1	8/10 · 8/20 · 2(.8)(.4)/1.2	0.80 · 0.40 · 0.533
Luck bar (N = 20, sd 0.5)	$0.5 \times \sqrt{2 \ln 20} = 0.5 \times 2.448$	1.22 vs reported 1.20 — luck-level

acc 98.6% vs dumb 98.0%

$F_1 = 0.533$

luck bar (N=20): $0.5\sqrt{2 \ln 20} = 1.22$

Both Headlines Died Under Audit — Good

That is stage 3 doing its job: better you kill them than the market does

0.6pp

The accuracy 'edge' over predicting nothing at all

0.533

F1 — the number the write-up should lead with

1.22

What 20 junk trials produce by chance alone

N

The variable every backtest must report — trials

The general lesson

- Every metric has a dumb baseline — always compute it first.
- Every search has a luck bar — it rises with $\sqrt{2 \ln N}$.

For your project

- Name your metric AND its baseline in the same sentence.
- Write your trial count next to your best score. Non-negotiable.

Scale It: the Two Audits on Real Data

Paste the metaprompt — Claude Code runs both deflations on the class dataset

[INPUT] Prompt · Claude Code

```
Build a single Colab-ready Python script that:  
1) verifies the toy first: TP 8, FP 2, FN 12, TN 978  
   -> accuracy 98.6% vs baseline 98.0%, precision  
   0.80, recall 0.40, F1 0.533; luck bar  
    $0.5 \cdot \sqrt{2 \ln 20} = 1.22$ ;  
   (reference: fml_w16_p1_confusion.csv)  
2) loads the class set (make_classification, 20000x20,  
   weights [0.99, 0.01], random_state=42), trains a  
   RandomForest with stratified 5-fold CV;  
3) reports accuracy vs the all-negative baseline, then  
   F1 and PR-AUC - show the honest gap;  
4) luck experiment: score 20 models on PURE-NOISE  
   labels; print the best CV score found and compare  
   to the honest model's - the 'discovery for free';  
5) three-sentence verdict: what the dumb baseline and  
   the luck bar each protect you from, in this  
   week's language.
```

[OUTPUT] What to expect

```
toy check: 0.533 / 1.22  
          exactly  
baseline gap: accuracy  
              flatters, F1 tells  
noise search: a 'best'  
              junk model that looks  
              respectable
```

```
Your pencil audit,  
promoted to a standing  
stage-3 harness.
```

P2

PROBLEM 2 • COSTS

Net of Costs, or Not at All

Four numbers separate a strategy from a donation to your broker

A Trading Track's Headline, Audited

The draft claims '12% a year'. Facts: edge 30bp per trade, cost 12bp per trade, 40 trades a year, volatility 10%

Input	Value
Gross edge per trade	30bp
All-in cost per trade (spread + fees + impact)	12bp
Trades per year	40
Strategy volatility	10% a year

Helper

- $\text{net} = (\text{edge} - \text{cost}) \times \text{trades}$; Sharpe = return / volatility.

Your deliverables

- Gross and net return; both Sharpes; breakeven cost; 20-trade net.

The Tool: Subtract Before You Celebrate

Costs scale with activity; the edge doesn't have to — that asymmetry decides careers

$$\text{net} = (\text{edge} - \text{cost}) \times \text{trades}_{\text{net}} = \frac{\text{net return}}{\text{volatility}}$$

Reading it

- Every trade pays the cost whether the edge shows up or not.
- Net Sharpe falls in exact proportion to the return haircut.
- Past breakeven cost, more activity means more loss.

Lab guide

- Step 1: gross = edge $\times$ trades; net = (edge – cost) $\times$ trades.
- Step 2: divide both by volatility for the two Sharpes.
- Step 3: breakeven cost = edge; redo net at 20 trades.

Five Lines From Applause to Honest

Two-fifths of the headline was the broker's

Line	Arithmetic	Value
Gross return	$30\text{bp} \times 40$	12.0%
Net return	$(30 - 12)\text{bp} \times 40$	7.2%
Sharpe, gross $\rightarrow$ net	$12/10 \rightarrow 7.2/10$	$1.20 \rightarrow 0.72$
Breakeven cost	= the edge	30bp
Halved turnover (20 trades)	$(30 - 12)\text{bp} \times 20$	3.6% — activity is half the story

gross 12% $\rightarrow$ net $(30 - 12)\text{bp} \times 40 = 7.2\%$ $SR : 1.2 \rightarrow 0.72$

12% Gross Is 7.2% Net — Say the Second Number

A strategy is what survives the costs, not what precedes them

4.8%

The annual toll: 12bp
× 40 trades, paid win
or lose

0.72

The net Sharpe — the
one your write-up
must lead with

30bp

Breakeven cost: above
it, trading is charity

× 40

Turnover — the
multiplier on every
cost you have

The general lesson

- Costs are certain; edges are hypotheses — subtract accordingly.
- W13's execution tools exist precisely to shrink the 12bp.

For your project

- Trading track: report gross AND net, with the cost assumption stated.
- A net-negative result honestly shown still passes — see the FAQ.

Scale It: a Cost-Aware Backtest Report

Paste the metaprompt — Claude Code sweeps costs and turnover and draws the survival map

[INPUT] Prompt · Claude Code

Build a single Colab-ready Python script that:

- 1) verifies the toy first: edge 30bp, cost 12bp, 40 trades, vol 10% -> gross 12%, net 7.2%, Sharpe 1.20 -> 0.72, breakeven 30bp, 20-trade net 3.6%;
- 2) simulates 5 years of daily strategy returns with a true 30bp-per-trade edge, 40 trades/yr, vol 10% (seeded numpy);
- 3) applies per-trade costs of 0, 6, 12, 20, 30, 40bp; plots the equity curves on one chart;
- 4) heatmap: net Sharpe over a grid of cost (0-40bp) x trades/yr (10-100); mark the breakeven frontier;
- 5) prints the exact (cost, turnover) pairs where the strategy dies;
- 6) three-sentence verdict: why 'net of costs' is a stage-5 honesty rule, in this week's language.

[OUTPUT] What to expect

toy check: 7.2% / 0.72 / 3.6% exactly
curves: fanning apart as costs rise
heatmap: a clean death frontier at edge = cost

Your five pencil lines, grown into the chart every PM asks for first.

P3

PROBLEM 3 • RUBRIC

Grade Like a Grader

Three projects, five weights — see exactly where the points live

Three Drafts Walk Into a Review

Stage scores 0–10. Weights: data .20, model .15, test .30, explain .20, present .15

Project	Data	Model	Test	Explain	Present
A · fancy model, thin testing	6	10	3	2	8
B · simple model, honest work	8	6	9	9	7
C · middle of the road	7	8	6	5	6

Your deliverables

- The weighted total for each project, and the ranking.
- One line each: where A lost it, and where B won it.

The Tool: a Weighted Sum That Encodes Values

The weights ARE the course's opinion — half the grade sits on honesty and explanation

$$\text{score} = \sum_k w_k s_k = (0.20, 0.15, 0.30, 0.20, 0.15) \text{ — test + explain carry 50\%}$$

Reading the weights

- Test (.30) is the single largest stage — stage 3 is the grade.
- Model (.15) is the SMALLEST — the box everyone overworks.
- Explain (.20) + test = 50%: trust is half the mark.

Lab guide — per project

- Step 1: multiply each stage score by its weight.
- Step 2: add the five products; two decimals.
- Step 3: rank; locate each project's biggest gain and loss.

Fifteen Products, Three Totals

B's honesty alone ($2.7 + 1.8$) nearly equals A's whole project

Project	Weighted arithmetic	Total
A · fancy	$0.2 \cdot 6 + 0.15 \cdot 10 + 0.3 \cdot 3 + 0.2 \cdot 2 + 0.15 \cdot 8 = 1.2 + 1.5 + 0.9 + 0.4 + 1.2$	5.20
B · honest	$0.2 \cdot 8 + 0.15 \cdot 6 + 0.3 \cdot 9 + 0.2 \cdot 9 + 0.15 \cdot 7 = 1.6 + 0.9 + 2.7 + 1.8 + 1.05$	8.05
C · middle	$0.2 \cdot 7 + 0.15 \cdot 8 + 0.3 \cdot 6 + 0.2 \cdot 5 + 0.15 \cdot 6 = 1.4 + 1.2 + 1.8 + 1.0 + 0.9$	6.30

A (fancy): 5.20

B (simple, honest): 8.05 ✓

C (middle): 6.30

B Wins by 2.85 With the Plainest Model in the Room

A's model score of 10 bought 1.5 points; its testing score of 3 threw away 2.1

8.05

B — simple, honest,
explained: the A grade

5.20

A — a 10/10 model
drowned by a 3/10
test

4.5

B's points from test +
explain alone (of 5
possible)

.15

The model's weight —
the smallest box on
the board

The general lesson

- Rubrics are value statements: this one pays for trust.
- The same weighting runs every real model-risk review.

For your project

- Self-score honestly with this table before submitting.
- Your cheapest points: move test and explain up one notch each.

Scale It: an Automated Pre-Flight Review

Paste the metaprompt — Claude Code audits your actual project against gates and rubric

[INPUT] Prompt · Claude Code

You are my final-project pre-flight reviewer. My project code and write-up are in this repo/notebook.

- 1) Verify the toy rubric first: weights (.20, .15, .30, .20, .15); scores A (6,10,3,2,8), B (8,6,9,9,7), C (7,8,6,5,6) -> totals 5.20, 8.05, 6.30; (reference: fml_w16_p3_rubric.csv)
- 2) audit my project against the four gates - leakage, luck (count my trials from the code), explanation, per-group fairness - citing the exact cell/line where each passes or fails;
- 3) score my five stages 0-10 with one sentence of evidence each; compute my weighted total;
- 4) list my three cheapest points: the smallest changes that raise the total most;
- 5) write the two toughest examiner questions my draft invites, each with the best answer it currently supports.

[OUTPUT] What to expect

toy check: 5.20 / 8.05 /
6.30 exactly
gate audit: pass/fail
with line citations
your total, with the
evidence trail
cheapest points: usually
in stages 3 and 4

The grader's table,
turned into your own
pre-flight instrument.

Problem Set — Three Things Your Hands Now Know

The final set was three audits — run all three on your own project this week

1

Accuracy 98.6% deflated to F1 0.533; the 1.2 met the 1.22 luck bar

Every metric has a dumb baseline and every search has a luck bar — compute both before believing anything.

2

12% gross became 7.2% net — Sharpe 1.20 → 0.72

Costs are certain, edges are hypotheses. Report net, state the cost assumption, and respect turnover.

3

The honest simple project scored 8.05; the fancy one 5.20

Test + explain carry half the rubric. Your cheapest points are in honesty, not in a bigger model.

Sixteen weeks of pencil-first discipline — now aim it at your own work. Go build.